



|                                                     |                                                                                                                     |
|-----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Unidad IV. Desarrollo de Aplicaciones web dinámicas | ACTIVIDAD: DSW-A14. Despliegue seguro de aplicación Node.js MVC con Apache, HTTPS y cabeceras de seguridad en Azure |
| TIPO: Actividad individual                          |                                                                                                                     |

## Objetivo

El estudiante configurará un despliegue seguro para una aplicación web Node.js MVC en una máquina virtual Linux en Azure, utilizando Apache HTTP Server como proxy inverso, HTTPS en el puerto 443, certificados TLS, PM2 para ejecución persistente y mecanismos básicos de seguridad HTTP.

## Descripción

En esta práctica, el estudiante retomará la aplicación desplegada en la actividad DSW-A13 y mejorará su configuración para aproximarla a un entorno de producción. En lugar de exponer directamente la aplicación Node.js por el puerto 3000, Apache recibirá las solicitudes HTTP/HTTPS y las redirigirá internamente hacia Node.js mediante un proxy inverso.

Apache permite configurar proxy inverso mediante directivas como ProxyPass y ProxyPassReverse. Además, se utilizará Certbot para configurar HTTPS con Apache, ya que Certbot puede editar automáticamente la configuración de Apache para servir el sitio mediante HTTPS.

También se incorporarán cabeceras de seguridad tanto en Apache como en Express. En Express usará Helmet, middleware que configura cabeceras HTTP de seguridad como Content-Security-Policy y Strict-Transport-Security.

## Material y equipo requeridos

- a) Máquina virtual Ubuntu en Azure configurada en DSW-A13
- b) Aplicación Node.js MVC desplegada y funcionando
- c) Acceso SSH a la VM
- d) UFW configurado en Ubuntu
- e) Navegador web actualizado

## Resultados Entregables

El estudiante deberá entregar un archivo PDF con evidencias del despliegue seguro de la aplicación Node.js MVC utilizando Apache, HTTPS y mecanismos de endurecimiento básico del servidor.

El documento deberá incluir las siguientes evidencias y explicaciones:

### 1. Evidencias de infraestructura Azure

- a) Captura de la máquina virtual creada en Azure.
- b) Captura de la IP pública asociada a la VM.



c) Captura del nombre DNS configurado en Azure (cloudapp.azure.com).

d) Captura de las reglas NSG habilitadas para:

- SSH (22)
- HTTP (80)
- HTTPS (443)

## **2. Evidencias de Apache como proxy inverso**

a) Captura del servicio Apache activo:

```
sudo systemctl status apache2
```

b) Captura del archivo:

```
/etc/apache2/sites-available/dsw-a14.conf
```

c) Captura del archivo HTTPS generado por Certbot:

```
/etc/apache2/sites-available/dsw-a14-le-ssl.conf
```

d) Explicación breve del funcionamiento del proxy inverso implementado.

## **3. Evidencias de HTTPS y Certbot**

a) Captura de ejecución exitosa de:

```
sudo certbot --apache
```

b) Captura de la validación:

```
sudo certbot renew --dry-run
```

c) Captura del sitio funcionando mediante:

```
https://dominio.cloudapp.azure.com
```

Con la consola libre de errores

d) Captura del detalle del certificado HTTPS en el navegador.

e) Explicación breve de la función de Certbot y Let's Encrypt.



#### **4. Evidencias de PM2 y Node.js**

a) Captura de:

```
pm2 status
```

b) Captura de:

```
pm2 logs
```

c) Explicación breve de por qué PM2 es importante en despliegues Node.js.

#### **5. Evidencias de seguridad y endurecimiento**

a) Captura de las cabeceras de seguridad configuradas en Apache.

b) Captura del código donde se implementó Helmet en index.js.

c) Captura de la política CSP configurada.

d) Explicación breve de:

- Qué es Helmet
- Qué es CSP
- Y por qué se ajustó CSP para permitir Bootstrap CDN y recursos externos

#### **6. Evidencias de firewall y cierre del puerto 3000**

a) Captura de:

```
sudo ufw status
```

b) Evidencia de que el puerto 3000 ya no está expuesto externamente.

c) Explicación breve de por qué es más seguro utilizar Apache + HTTPS en lugar de exponer directamente Node.js.

#### **7. Validaciones finales**

a) Captura de:

```
curl -I https://dominio.cloudapp.azure.com
```

Donde se observen cabeceras como:

- Strict-Transport-Security
- X-Frame-Options
- Content-Security-Policy



b) Captura de:

```
sudo ss -tulpn
```

c) Explicación breve del significado de las cabeceras observadas.

## 9. Reflexión final

El estudiante deberá redactar una reflexión final donde explique:

- Qué aprendió durante el despliegue
- Dificultades encontradas
- Diferencias entre un entorno académico y un entorno de producción real

## PROCEDIMIENTO

### PARTE A. Instalación de PM2

#### Paso 1 – Instalar PM2

PM2 (Process Manager 2), es un administrador de procesos de Node.js. permite mantener aplicaciones web ejecutándose en segundo plano (24/7), reiniciarlas automáticamente si fallan y balancear la carga. Es decir, la aplicación continuará ejecutándose aunque se cierre la sesión SSH.

Instala PM2 globalmente:

```
sudo npm install -g pm2
```

Verifica:

```
pm2 -v
```

#### Paso 2 – Ejecutar la aplicación con PM2

Desde la carpeta del proyecto:

```
cd /var/www/dsw-a13  
pm2 start index.js --name dsw-a13-app
```



```
azureuser@vm-dsw-a13-2-ErikaMeneses:/var/www/dsw-a13$ pm2 start index.js --name dsw-a13-app
```

```
-----
      /\      /\      /\      /\      /\
     /\  /\  /\  /\  /\  /\  /\  /\  /\
    /\ /\ /\ /\ /\ /\ /\ /\ /\ /\ /\
   /\ /\ /\ /\ /\ /\ /\ /\ /\ /\ /\
  /\ /\ /\ /\ /\ /\ /\ /\ /\ /\ /\
 /\ /\ /\ /\ /\ /\ /\ /\ /\ /\ /\
/\ /\ /\ /\ /\ /\ /\ /\ /\ /\ /\
-----

Runtime Edition

PM2 is a Production Process Manager for Node.js applications
with a built-in Load Balancer.

Start and Daemonize any application:
$ pm2 start app.js

Load Balance 4 instances of api.js:
$ pm2 start api.js -i 4

Monitor in production:
$ pm2 monitor

Make pm2 auto-boot at server restart:
$ pm2 startup

To go further checkout:
http://pm2.io/

-----

[PM2] Spawning PM2 daemon with pm2_home=/home/azureuser/.pm2
[PM2] PM2 Successfully daemonized
[PM2] Starting /var/www/dsw-a13/index.js in fork_mode (1 instance)
[PM2] Done.
```

| id | name        | namespace | version | mode | pid   | uptime | σ | status | cpu | mem    | user   | watching |
|----|-------------|-----------|---------|------|-------|--------|---|--------|-----|--------|--------|----------|
| 0  | dsw-a13-app | default   | 1.0.0   | fork | 15846 | 0s     | 0 | online | 0%  | 13.8mb | azu... | disabled |

Verifica el estado:

```
pm2 status
```

Consulta logs:

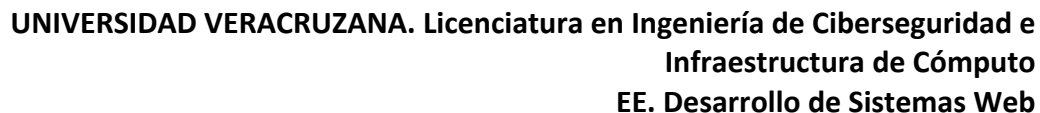
```
pm2 logs dsw-a13-app
```

### Paso 3 – Configurar PM2 para reinicio automático

Guarda el proceso actual:

```
pm2 save
```

Configura PM2 para iniciar automáticamente cuando reinicie la VM:



PM2 mostrará un comando largo con sudo. Cópialo y ejecútalo.

Después vuelve a guardar:

## Paso 4 – Verificar que la aplicación sigue activa

Cierra la sesión SSH:

Desde el navegador abre:

La aplicación debe seguir funcionando.

## Paso 5 – Comandos útiles de PM2

Ver aplicaciones activas:



```
pm2 list
```

Reiniciar aplicación:

```
pm2 restart dsw-a13-app
```

Detener aplicación:

```
pm2 stop dsw-a13-app
```

Eliminar aplicación de PM2:

```
pm2 delete dsw-a13-app
```

Ver logs:

```
pm2 logs
```

## PARTE B. Configuración de Apache como proxy inverso y preparación del entorno HTTPS

### Paso 1 – Conectarse a la máquina virtual por SSH

Desde tu equipo local abre una terminal y ejecuta:

```
ssh -i ruta/llave.pem azureuser@IP_PUBLICA
```

Ejemplo:

```
ssh -i ~/Downloads/vm-dsw-a13.pem azureuser@20.120.55.10
```

Si la conexión es exitosa aparecerá el prompt de Ubuntu.

### Paso 2 – Verificar estado actual de la aplicación Node.js

Verifica que la aplicación siga funcionando mediante PM2:

```
pm2 status
```

Debe aparecer algo similar a:



| id | name        | status | cpu |
|----|-------------|--------|-----|
| 0  | dsw-a13-app | online | 0%  |

También verifica que Node.js esté escuchando en el puerto 3000:

```
sudo ss -tulpn | grep 3000
```

### Paso 3 – Instalar Apache HTTP Server

Actualiza paquetes:

```
sudo apt update
```

Instala Apache:

```
sudo apt install -y apache2
```

Verifica el estado del servicio:

```
sudo systemctl status apache2
```

Debe aparecer:

active (running)

```
no VM guests are running extended hypervisor (qemu) binaries on this host.
[azureuser@vm-dsw-a13-2-ErikaMeneses:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
  Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
  Active: active (running) since Sun 2026-05-24 05:59:57 UTC; 27s ago
    Docs: https://httpd.apache.org/docs/2.4/
  Main PID: 36111 (apache2)
    Tasks: 55 (limit: 2245)
  Memory: 5.2M (peak: 5.5M)
    CPU: 46ms
  CGroup: /system.slice/apache2.service
          └─36111 /usr/sbin/apache2 -k start
             └─36114 /usr/sbin/apache2 -k start
                └─36115 /usr/sbin/apache2 -k start

May 24 05:59:57 vm-dsw-a13-2-ErikaMeneses systemd[1]: Starting apache2.service - The Apache HTTP Server...
May 24 05:59:57 vm-dsw-a13-2-ErikaMeneses systemd[1]: Started apache2.service - The Apache HTTP Server.
[azureuser@vm-dsw-a13-2-ErikaMeneses:~$
```





## Paso 4 – Abrir puertos HTTP y HTTPS en Azure

En el portal de Azure:

1. Abre la máquina virtual.
2. Selecciona:

Networking

3. Verifica reglas de entrada para:

| Puerto | Protocolo | Acción |
|--------|-----------|--------|
| 80     | TCP       | Allow  |
| 443    | TCP       | Allow  |

Reglas [Contraer todo](#)

Grupo de seguridad de red **vm-dsw-a13-2-ErikaMeneses-nsg** (conectado a networkInterface: **vm-dsw-a13-2-erikameneses468**)  
Afecta a 0 subredes, 1 interfaces de red

+ Crear ACL del puerto

Regla de puerto de entrada

Regla de puerto de salida

Buscar reglas

Origen == todo

Destino == todo

Protocolo == todo

Acción == todo

Puerto == todo

| Prio...                         | Nombre           |  | Puerto     | Protocolo  | Origen         | Destino        | Acción |
|---------------------------------|------------------|--|------------|------------|----------------|----------------|--------|
| Reglas de puerto de entrada (6) |                  |  |            |            |                |                |        |
| 300                             | SSH              |  | 22         | TCP        | Cualquiera     | Cualquiera     | Allow  |
| 310                             | Allow-Node-3000  |  | 3000       | TCP        | Cualquiera     | Cualquiera     | Allow  |
| 320                             | HTTP             |  | 80         | TCP        | Cualquiera     | Cualquiera     | Allow  |
| 65000                           | AllowVnetInBound |  | Cualquiera | Cualquiera | VirtualNetwork | VirtualNetwork | Allow  |

Si no existen:

Agregar HTTP

| Campo                   | Valor |
|-------------------------|-------|
| Destination port ranges | 80    |
| Protocol                | TCP   |
| Action                  | Allow |

Agregar HTTPS

| Campo                   | Valor |
|-------------------------|-------|
| Destination port ranges | 443   |
| Protocol                | TCP   |



|        |       |
|--------|-------|
| Action | Allow |
|--------|-------|

 **Agregar regla de seguridad de entrada** ✕

vm-dsw-a13-2-ErikaMeneses-nsg

Origen ⓘ  

Any

Intervalos de puertos de origen \* ⓘ  

\*

Destino ⓘ  

Any

Servicio ⓘ  

HTTPS

Intervalos de puertos de destino ⓘ  

443

Protocolo  

☐ Any

☒ TCP

☐ UDP

☐ ICMPv4

Agregar

Cancelar

 Enviar comentarios

## Paso 5 – Configurar UFW para Apache

Dentro de Ubuntu:

```
sudo ufw allow 'Apache Full'
```

Verifica:

```
sudo ufw status
```

Debe aparecer:

```
Apache Full ALLOW Anywhere
```



```
[azureuser@vm-dsw-a13-2-ErikaMeneses:~]$ sudo ufw allow 'Apache Full'
Rule added
Rule added (v6)
[azureuser@vm-dsw-a13-2-ErikaMeneses:~]$ sudo ufw status
Status: active
```

| To               | Action | From          |
|------------------|--------|---------------|
| OpenSSH          | ALLOW  | Anywhere      |
| 80/tcp           | ALLOW  | Anywhere      |
| 3000/tcp         | ALLOW  | Anywhere      |
| Apache Full      | ALLOW  | Anywhere      |
| OpenSSH (v6)     | ALLOW  | Anywhere (v6) |
| 80/tcp (v6)      | ALLOW  | Anywhere (v6) |
| 3000/tcp (v6)    | ALLOW  | Anywhere (v6) |
| Apache Full (v6) | ALLOW  | Anywhere (v6) |

## Paso 6 – Instalar módulos Apache necesarios

Habilita módulos para proxy inverso, HTTPS y cabeceras:

```
sudo a2enmod proxy
sudo a2enmod proxy_http
sudo a2enmod ssl
sudo a2enmod headers
sudo a2enmod rewrite
```

Reinicia Apache:

```
sudo systemctl restart apache2
```

```
[azureuser@vm-dsw-a13-2-ErikaMeneses:~]$ sudo a2enmod proxy
Command 'sudo' not found, did you mean:
  command 'sudo' from deb sudo (1.9.15p5-3ubuntu5.24.04.2)
  command 'sudo' from deb sudo-ldap (1.9.15p5-3ubuntu5.24.04.2)
Try: sudo apt install <deb name>
[azureuser@vm-dsw-a13-2-ErikaMeneses:~]$ a2enmod proxy
Could not create /etc/apache2/mods-enabled/proxy.conf: Permission denied
[azureuser@vm-dsw-a13-2-ErikaMeneses:~]$ sudo a2enmod proxy
Enabling module proxy.
To activate the new configuration, you need to run:
  systemctl restart apache2
[azureuser@vm-dsw-a13-2-ErikaMeneses:~]$ sudo a2enmod proxy_http
Considering dependency proxy for proxy_http:
Module proxy already enabled
Enabling module proxy_http.
To activate the new configuration, you need to run:
  systemctl restart apache2
[azureuser@vm-dsw-a13-2-ErikaMeneses:~]$ sudo a2enmod ssl
Considering dependency mime for ssl:
Module mime already enabled
Considering dependency socache_shmcb for ssl:
Enabling module socache_shmcb.
Enabling module ssl.
See /usr/share/doc/apache2/README.Debian.gz on how to configure SSL and create self-signed certificates.
To activate the new configuration, you need to run:
  systemctl restart apache2
[azureuser@vm-dsw-a13-2-ErikaMeneses:~]$ sudo a2enmod headers
Enabling module headers.
To activate the new configuration, you need to run:
  systemctl restart apache2
[azureuser@vm-dsw-a13-2-ErikaMeneses:~]$ sudo a2enmod rewrite
Enabling module rewrite.
To activate the new configuration, you need to run:
  systemctl restart apache2
[azureuser@vm-dsw-a13-2-ErikaMeneses:~]$ sudo systemctl restart apache2
[azureuser@vm-dsw-a13-2-ErikaMeneses:~]$
```



## **PARTE C. Obtener un dominio DNS gratuito con Azure (si es el proveedor de tu IP pública) o DuckDNS**

Si tu proveedor de dominio es Microsoft Azure, conviene que configures el dominio DNS que Azure te proporciona de forma gratuita con la IP pública contratada, si tienes otro proveedor que no te ofrece un nombre de dominio puedes usar otros servicios como DuckDNS para generar un dominio DNS gratuito.

### **Opción 1. Configurar un dominio DNS gratuito en Azure**

Azure permite asociar un nombre DNS gratuito a una dirección IP pública.

Cuando se configura un:  
DNS Name Label

Azure crea automáticamente un registro DNS que apunta hacia la IP pública de la máquina virtual.

El resultado es un dominio con formato:  
nombre.region.cloudapp.azure.com

Ejemplo:  
dsw-erika.eastus.cloudapp.azure.com

Y este nombre puede utilizarse para:

- Acceder al servidor web
- Configurar https
- Generar certificados tls
- Evitar depender directamente de la IP pública

### **Paso 1 – Abrir la configuración de la IP pública**

En el portal de Azure:

1. Abre la máquina virtual.
2. En la sección:

Overview

Localiza:

Public IP address



Ayuda para copiar esta máquina virtual en cualquier región

Conectar ▾ ▶ Iniciar ↺ Reiniciar □ Detener ⌚ Hibernar 📷 Captura ▾ 🗑 Eliminar ↻ Actualizar ↕ Escalar

#### ^ Información esencial

Grupo de recursos [\(mover\)](#)  
[rg-dsw-a13 2-ErikaMeneses](#)

Estado  
En ejecución

Ubicación  
East US

Suscripción [\(mover\)](#)  
[Suscripción de Azure Erika](#)

Id. de suscripción  
e00f8f21-6a83-422d-9f11-83581bcb44f5

Sistema operativo  
Linux (ubuntu 24.04)

Tamaño  
Standard B1ms (1 vcpu, 2 GiB de memoria)

IP pública de NIC principal  
[104.211.57.187](#) 📄

[1 direcciones IP públicas asociadas](#)

Red virtual/subred  
[vm-dsw-a13-2-ErikaMeneses-vnet/default](#)

Nombre DNS  
[Sin configurar](#)

Estado de mantenimiento

3. Haz clic sobre la IP pública.

Azure abrirá el recurso:

Public IP Address

## Paso 2 – Configurar DNS Name Label

En el recurso Public IP Address busca:

DNS name label

↻ Actualizar

#### Resumen

Asignación Static

Dirección IP pública ⓘ 104.211.57.187

Tiempo de espera de inactividad (minutos) ⓘ

Etiqueta de nombre DNS (opcional)



Escribe un nombre único.

Ejemplo:

dsw-erika

### Paso 3 – Guardar configuración

Haz clic en:

Save

Azure generará automáticamente un FQDN (Fully Qualified Domain Name) similar a:

dsw-erika.eastus.cloudapp.azure.com

La región puede variar según la ubicación seleccionada para la VM.

### Paso 4 – Verificar el FQDN generado

Regresa a:

Overview

Verifica que ahora aparezca:

DNS name

Ejemplo:

dsw-erika.eastus.cloudapp.azure.com

#### ^ Información esencial

Grupo de recursos ([mover](#))  
[rg-dsw-a13\\_2-ErikaMeneses](#)

Estado  
En ejecución

Ubicación  
East US

Suscripción ([mover](#))  
[Suscripción de Azure Erika](#)

Id. de suscripción  
e00f8f21-6a83-422d-9f11-83581bcb44f5

Sistema operativo  
Linux (ubuntu 24.04)

Tamaño  
Standard B1ms (1 vcpu, 2 GiB de memoria)

IP pública de NIC principal  
[104.211.57.187](#)

[1 direcciones IP públicas asociadas](#)

Red virtual/subred  
[vm-dsw-a13-2-ErikaMeneses-vnet/default](#)

Nombre DNS  
[dsw-erika.eastus.cloudapp.azure.com](#)

Estado de mantenimiento  
-



## Paso 5 – Verificar resolución DNS

Desde Ubuntu ejecuta:

```
ping dsw-erika.eastus.cloudapp.azure.com
```

o:

```
nslookup dsw-erika.eastus.cloudapp.azure.com
```

El dominio debe resolver hacia la IP pública de Azure.

```
[azureuser@vm-dsw-a13-2-ErikaMeneses:~$ nslookup dsw-erika.eastus.cloudapp.azure.com
Server:      127.0.0.53
Address:     127.0.0.53#53

Non-authoritative answer:
Name:   dsw-erika.eastus.cloudapp.azure.com
Address: 104.211.57.187
```

¿Por qué necesitamos un dominio?

Let's Encrypt emite certificados TLS válidos únicamente para dominios o subdominios.

HTTPS requiere:

- Un nombre DNS
- Validación del dominio
- Y un certificado digital

Importante:

Para que el nombre DNS no cambie, la IP pública de Azure debe configurarse como estática.

Cómo verificar IP estática

En:

Public IP → Configuration

Debe decir:

Assignment: Static

Si aparece:

Dynamic



Cambiar a:  
Static

y Guardar.

## **Opción 2. Crear un dominio DNS gratuito en DuckDNS**

### **Paso 1 – Crear cuenta en DuckDNS**

Abre:

[DuckDNS](https://duckdns.org/)

Inicia sesión usando:

- GitHub,
- Google,
- Reddit,
- Twitter,
- o Microsoft.

### **Paso 2– Crear un subdominio**

En el campo:  
domains

Escribe un nombre.

Ejemplo:  
dsw-erika

Haz clic en:  
add domain

Obtendrás:  
dsw-erika.duckdns.org

### **Paso 3 – Asociar la IP pública de Azure u otro proveedor**

En el portal Azure copia la IP pública de la VM.

Ejemplo:

20.120.55.10





En DuckDNS:

1. Pega la IP.
2. Haz clic en:

`update ip`

#### Paso 4 – Verificar resolución DNS

Desde Ubuntu ejecuta:

```
ping dsw-erika.duckdns.org
```

Debe responder con la IP pública Azure.

También puedes usar:

```
nslookup dsw-erika.duckdns.org
```

#### PARTE D. Configurar Apache como proxy inverso

En esta práctica Apache funcionará como proxy inverso. Un proxy inverso es un servidor intermedio que recibe las solicitudes de los usuarios desde Internet y las redirige internamente hacia otro servicio o aplicación, en este caso, hacia la aplicación Node.js ejecutándose en el puerto 3000.

La arquitectura será:

```
Internet
  ↓
Apache (puerto 80 / 443)
  ↓
Node.js (localhost:3000)
```

En lugar de exponer directamente Node.js al exterior mediante el puerto 3000, los usuarios únicamente se comunicarán con Apache utilizando HTTP o HTTPS.

Ventajas de esta arquitectura

- Apache administra HTTPS
- Node.js no se expone directamente a Internet
- Permite agregar seguridad adicional
- Facilita escalabilidad y balanceo
- Reduce exposición del puerto 3000



En este escenario, Node.js funcionará únicamente como servidor de aplicación, mientras que Apache actuará como servidor web frontal encargado de manejar el tráfico externo de manera más segura y controlada.

### **Paso 1 – Crear configuración del sitio**

Crea archivo:

```
sudo nano /etc/apache2/sites-available/dsw-a14.conf
```

Agrega:

```
<VirtualHost *:80>

    ServerName dsw-erika.eastus.cloudapp.azure.com

    ProxyPreserveHost On

    ProxyPass / http://127.0.0.1:3000/
    ProxyPassReverse / http://127.0.0.1:3000/

    ErrorLog ${APACHE_LOG_DIR}/dsw-a14-error.log
    CustomLog ${APACHE_LOG_DIR}/dsw-a14-access.log combined

</VirtualHost>
```

Guarda:

```
Ctrl + O
Enter
Ctrl + X
```

En el código agregado:

ServerName: Indica el dominio del sitio.

ProxyPass: Redirige solicitudes hacia Node.js.

ProxyPassReverse: Reescribe respuestas para mantener coherencia de URLs.

127.0.0.1:3000

Node.js escucha únicamente localmente, no desde Internet.



## Paso 2 – Habilitar el sitio

```
sudo a2ensite dsw-a14.conf
```

Deshabilita sitio por defecto:

```
sudo a2dissite 000-default.conf
```

Verifica configuración:

```
sudo apachectl configtest
```

Debe decir:

```
Syntax OK
```

Recarga Apache:

```
sudo systemctl reload apache2
```

```
[azureuser@vm-dsw-a13-2-ErikaMeneses:~$ sudo a2ensite dsw-a14.conf
Enabling site dsw-a14.
To activate the new configuration, you need to run:
  systemctl reload apache2
[azureuser@vm-dsw-a13-2-ErikaMeneses:~$ sudo a2dissite 000-default.conf
Site 000-default disabled.
To activate the new configuration, you need to run:
  systemctl reload apache2
[azureuser@vm-dsw-a13-2-ErikaMeneses:~$ sudo apachectl configtest
Syntax OK
[azureuser@vm-dsw-a13-2-ErikaMeneses:~$ sudo systemctl reload apache2
```

## Paso 3 – Probar el proxy inverso

En navegador abre tu URL, por ejemplo:

<http://dsw-erika.eastus.cloudapp.azure.com/>

La aplicación debe cargar sin usar :3000



No seguro

dsw-erika.eastus.cloudapp.azure.com

☆

🔖

👤

⋮

📁 AccesosRápidos

📁 Musica

📁 Doctorado

📁 Tesistas

📁 Eventos

📁 Material

Todos los favoritos

Universidad Veracruzana

# Alumnos con materias

## Lista de alumnos

Id: 1

Sofía Martínez López

Materia: Desarrollo de Sistemas Web

Promedio: 9.2

Detalles

Id: 2

Miguel Ángel Torres Ruiz

Materia: Enrutamiento Avanzado

Promedio: 8.5

Detalles

Id: 3

Carolina Herrera Gutiérrez

Materia: Bases de Datos

Promedio: 9.8

Detalles

Id: 4

Laura González Sánchez

Materia: Cómputo en la Nube

Promedio: 8

Detalles

Id: 5

Ricardo Fernández Moreno

Materia: Enrutamiento Avanzado

Promedio: 7.8

Detalles

Id: 6

Ana María Sánchez Martín

Materia: Arquitecturas de Red

Promedio: 9.5

Detalles

Id: 7

Juan Carlos Navarro Jiménez

Materia: Arquitecturas de Red

Promedio: 6.9

Detalles

Id: 8

Beatriz Molina Castro

Materia: Sistemas Operativos

Promedio: 8.3

Detalles

Id: 9

Fernando Gómez Velasco

Materia: Sistemas Operativos

Promedio: 7.6

Id: 10

José Luis Rodríguez Pérez

Materia: Estructuras de Datos

Promedio: 7.4

Id: 11

Claudia Jiménez Rivas

Materia: Estructuras de Datos

Promedio: 9.1

## Parte E. Configurar HTTPS con Let's Encrypt

### Paso 1 – Instalar Certbot

Certbot es una herramienta automática utilizada para obtener, instalar y renovar certificados digitales TLS/SSL emitidos por Let's Encrypt.

En esta práctica, Certbot permitirá configurar HTTPS en Apache de manera automática, habilitando conexiones seguras mediante el puerto 443.

Certbot automatiza tareas como:

- Solicitar certificados digitales
- Validar el dominio
- Configurar Apache para HTTPS
- Renovar certificados automáticamente
- Habilitar redirecciones seguras de HTTP hacia HTTPS

Así, Let's Encrypt es la autoridad certificadora gratuita y Certbot la herramienta que administra los certificados.



```
sudo apt install -y snapd  
sudo snap install core  
sudo snap refresh core  
sudo snap install --classic certbot
```

```
azureuser@vm-dsw-a13-2-ErikaMeneses:~$ sudo apt install -y snapd  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
Suggested packages:  
  zenity | kdialog  
The following packages will be upgraded:  
  snapd  
1 upgraded, 0 newly installed, 0 to remove and 5 not upgraded.  
Need to get 35.1 MB of archives.  
After this operation, 291 kB of additional disk space will be used.  
Get:1 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 snapd amd64 2.75.2+ubuntu24.04 [35.1 MB]  
Fetched 35.1 MB in 3s (13.1 MB/s)  
(Reading database ... 108934 files and directories currently installed.)  
Preparing to unpack .../snapd_2.75.2+ubuntu24.04_amd64.deb ...  
Unpacking snapd (2.75.2+ubuntu24.04) over (2.74.1+ubuntu24.04.4) ...  
Setting up snapd (2.75.2+ubuntu24.04) ...  
Installing new version of config file /etc/apparmor.d/usr.lib.snapd.snap-confine.real ...  
snapd.failure.service is a disabled or a static unit not running, not starting it.  
snapd.gpio-chardev-setup.target is a disabled or a static unit not running, not starting it.  
snapd.snap-repair.service is a disabled or a static unit not running, not starting it.  
Processing triggers for dbus (1.14.10-4ubuntu4.1) ...  
Processing triggers for man-db (2.12.0-4build2) ...  
Scanning processes...  
Scanning linux images...  
  
Running kernel seems to be up-to-date.  
  
No services need to be restarted.  
  
No containers need to be restarted.  
  
No user sessions are running outdated binaries.  
  
No VM guests are running outdated hypervisor (qemu) binaries on this host.  
azureuser@vm-dsw-a13-2-ErikaMeneses:~$ sudo snap install core  
2026-05-25T16:24:06Z INFO Waiting for automatic snapd restart...  
2026-05-25T16:24:07Z INFO Waiting for automatic snapd restart...  
2026-05-25T16:24:08Z INFO Waiting for automatic snapd restart...  
2026-05-25T16:24:09Z INFO Waiting for automatic snapd restart...  
core 16-2.61.4-20260225 from Canonical installed  
azureuser@vm-dsw-a13-2-ErikaMeneses:~$ sudo snap refresh core  
snap "core" has no updates available  
azureuser@vm-dsw-a13-2-ErikaMeneses:~$ sudo snap install --classic certbot  
2026-05-25T16:24:41Z INFO Waiting for automatic snapd restart...  
2026-05-25T16:24:42Z INFO Waiting for automatic snapd restart...  
2026-05-25T16:24:43Z INFO Waiting for automatic snapd restart...  
certbot 5.6.0 from Certbot Project (certbot-eff) installed
```

Crear enlace:

```
sudo ln -s /snap/bin/certbot /usr/bin/certbot
```

## Paso 2 – Obtener certificado HTTPS

En este paso se obtendrá el certificado y aplicará a tu dominio:

Ejecuta:

```
sudo certbot --apache
```

El asistente solicitará:

Correo electrónico  
Ingresa tu correo.



Aceptar términos

Selecciona: Y

Seleccionar dominio

Ejemplo: dsw-erika.eastus.cloudapp.azure.com/

Redirección HTTPS

Selecciona: 2: Redirect

Esto forzará HTTPS automáticamente.

```
azureuser@vm-dsw-a13-2-ErikaMeneses:~$ sudo certbot --apache
Saving debug log to /var/log/letsencrypt/letsencrypt.log
Enter email address or hit Enter to skip.
[ (Enter 'c' to cancel): ermeneses@uv.mx

-----
Please read the Terms of Service at:
https://letsencrypt.org/documents/LE-SA-v1.6-August-18-2025.pdf
You must agree in order to register with the ACME server. Do you agree?
-----
[(Y)es/(N)o: y

-----
Would you be willing, once your first certificate is successfully issued, to
share your email address with the Electronic Frontier Foundation, a founding
partner of the Let's Encrypt project and the non-profit organization that
develops Certbot? We'd like to send you email about our work encrypting the web,
EFF news, campaigns, and ways to support digital freedom.
-----
[(Y)es/(N)o: n
Account registered.

Which names would you like to activate HTTPS for?
We recommend selecting either all domains, or all domains in a VirtualHost/server block.
-----
1: dsw-erika.eastus.cloudapp.azure.com
-----
Select the appropriate numbers separated by commas and/or spaces, or leave input
blank to select all options shown (Enter 'c' to cancel): 1
Requesting a certificate for dsw-erika.eastus.cloudapp.azure.com

Successfully received certificate.
Certificate is saved at: /etc/letsencrypt/live/dsw-erika.eastus.cloudapp.azure.com/fullchain.pem
Key is saved at: /etc/letsencrypt/live/dsw-erika.eastus.cloudapp.azure.com/privkey.pem
This certificate expires on 2026-08-23.
These files will be updated when the certificate renews.
Certbot has set up a scheduled task to automatically renew this certificate in the background.

Deploying certificate
Successfully deployed certificate for dsw-erika.eastus.cloudapp.azure.com to /etc/apache2/sites-available/dsw-a14-le-ssl.conf
Congratulations! You have successfully enabled HTTPS on https://dsw-erika.eastus.cloudapp.azure.com

-----
If you like Certbot, please consider supporting our work by:
* Donating to ISRG / Let's Encrypt: https://letsencrypt.org/donate
* Donating to EFF: https://eff.org/donate-le
-----
```

Nota importante: Los certificados Let's Encrypt tienen una vigencia aproximada de 90 días.



### Paso 3 – Verificar renovación automática (simulación)

El siguiente comando simula todo el proceso de renovación del certificado SSL sin modificar realmente los archivos del sistema:

```
sudo certbot renew --dry-run
```

```
[azureuser@vm-dsw-a13-2-ErikaMeneses:~$ sudo certbot renew --dry-run
Saving debug log to /var/log/letsencrypt/letsencrypt.log

-----
Processing /etc/letsencrypt/renewal/dsw-erika.eastus.cloudapp.azure.com.conf
-----
Account registered.
Simulating renewal of an existing certificate for dsw-erika.eastus.cloudapp.azure.com

-----
Congratulations, all simulated renewals succeeded:
  /etc/letsencrypt/live/dsw-erika.eastus.cloudapp.azure.com/fullchain.pem (success)
-----
```

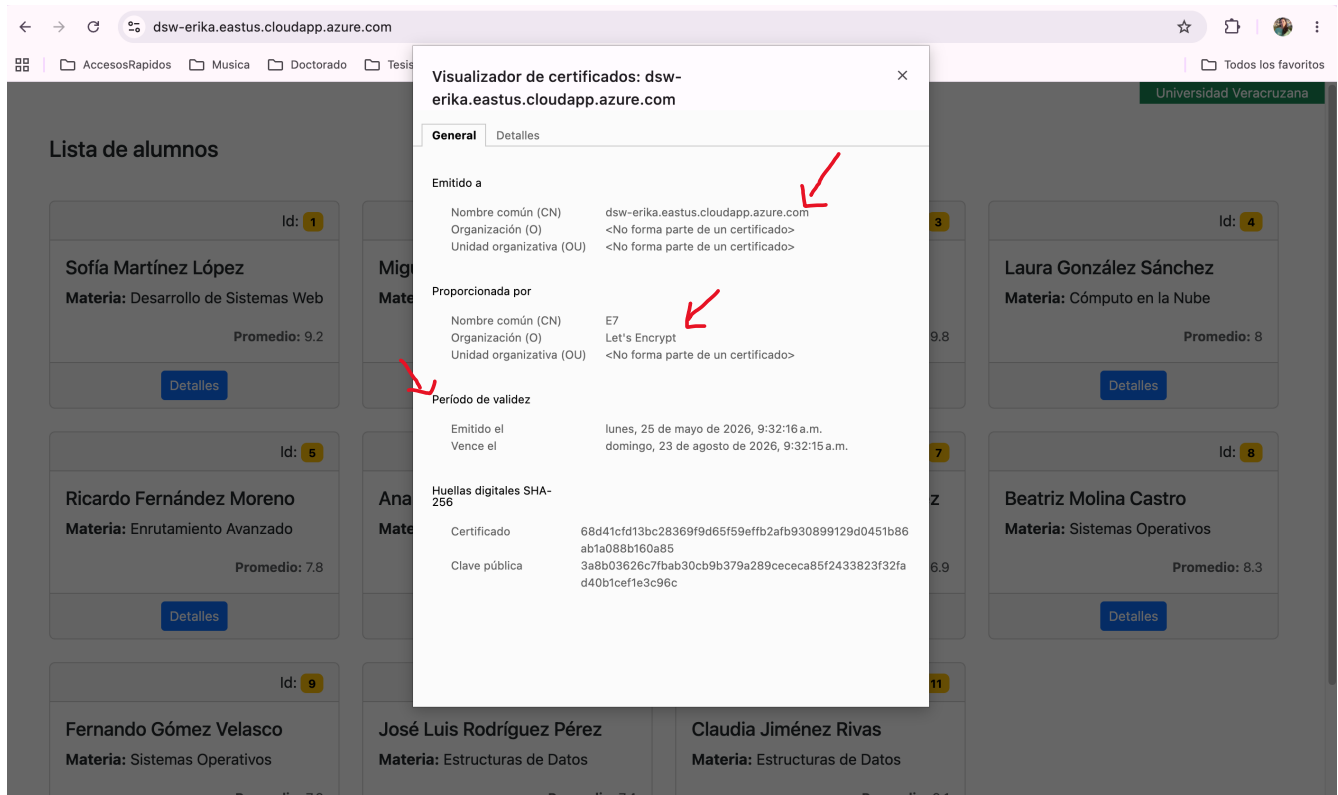
Si aparece el mensaje “Congratulations, all simulated renewals succeeded:”, estarás listo para renovar el certificado dentro de 90 días.

### Paso 4 – Verificar HTTPS

En navegador, por ejemplo:

<http://dsw-erika.eastus.cloudapp.azure.com/>

Debe aparecer el candado de seguridad y haciendo clic en el certificado, lo siguiente:



## Reforzamiento de seguridad mediante configuración de cabeceras de seguridad

Ahora se configurarán cabeceras de seguridad en dos niveles distintos:

- En la aplicación Node.js mediante Helmet.
- En Apache HTTP Server mediante directivas Header.

Esto se realiza porque ambos componentes cumplen funciones diferentes dentro de la arquitectura del sistema.

La arquitectura quedará así:

```
Usuario
↓
Apache HTTPS + cabeceras
↓
Node.js + Express + Helmet
↓
Aplicación MVC
```





Helmet es un middleware que funciona dentro de Node.js y Express, protege respuestas generadas por Express, agrega cabeceras desde la aplicación y aplica políticas relacionadas con el comportamiento del backend Node.js.

Apache funciona antes que Node.js, por lo que recibe primero las conexiones HTTPS, administra TLS, maneja certificados y puede agregar cabeceras incluso antes de que la solicitud llegue a Express. Por lo tanto, las cabeceras configuradas en Apache actúan a nivel de servidor web.

## **Parte F. Implementar Helmet en Express**

Helmet es un middleware para Express que agrega automáticamente múltiples cabeceras HTTP de seguridad.

Helmet ayuda a proteger contra ataques como:

- Cross-Site Scripting (XSS)
- Clickjacking
- MIME sniffing
- Inyección de contenido
- robo de información mediante iframes
- downgrade de HTTPS

### **Paso 1 – Instalar Helmet**

Dentro del proyecto:

```
cd /var/www/dsw-a13  
npm install helmet
```

### **Paso 2 – Modificar index.js**

Agregar:

```
const helmet = require("helmet");
```

Después de:

```
const express = require("express");
```

### **Paso 3 – Agregar middleware Helmet**

Antes de rutas y middlewares:



```
app.use(  
  helmet({  
    contentSecurityPolicy: false  
  })  
);
```

Se agregarán otros middleware que permiten servir CSS, javascript, etc. De tal forma que tu index.js debe quedar de la siguiente manera:

```
const express = require('express');  
const helmet = require('helmet');  
const dotenv = require('dotenv');  
  
const app = express();  
  
// Carga la configuración del archivo .env  
dotenv.config();  
  
// Obtiene el puerto desde .env  
const PORT = process.env.SERVER_PORT || 3000;  
  
// Configura EJS como motor de vistas  
app.set('view engine', 'ejs');  
  
// Middleware Helmet  
// Agrega cabeceras HTTP de seguridad automáticamente  
app.use(  
  helmet({  
    // Se desactiva CSP temporalmente  
    // porque Bootstrap y algunos recursos externos  
    // pueden ser bloqueados durante la práctica  
    contentSecurityPolicy: false  
  })  
);  
  
// Middleware para procesar formularios  
app.use(express.urlencoded({ extended: true }));  
  
// Middleware para archivos JSON  
app.use(express.json());  
  
// Middleware para archivos estáticos  
app.use(express.static('public'));  
  
// Rutas  
const homeRouter = require('./routes/home');  
  
app.use('/', homeRouter);  
app.use('/home', homeRouter);  
  
// Middleware para manejo de errores  
// Debe colocarse al final  
const errorHandler = require('./middlewares/errorhandler');  
  
app.use(errorHandler);
```



```
// Inicia el servidor web
app.listen(PORT, "0.0.0.0", () => {
  console.log(`Aplicación ejecutándose en el puerto ${PORT}`);
});
```

En este paso el archivo index.js debe ser modificado ya que Express procesa middlewares en orden. Helmet debe ejecutarse antes de enviar respuestas para poder agregar cabeceras HTTP de seguridad.

Helmet activa por defecto una política **Content Security Policy (CSP)** estricta. **CSP** es una cabecera de seguridad que restringe:

- Scripts
- Hojas de estilo
- Imágenes
- Fuentes
- iFrames
- Recursos externos

Ayuda a prevenir:

- XSS
- Inyección de scripts
- Carga de contenido malicioso

Sin embargo, durante esta práctica se utilizan recursos externos como:

- Bootstrap CDN
- Javascript externo
- Fuentes
- Estilos remotos

Una CSP estricta podría bloquear dichos recursos y hacer que la página no cargue correctamente.

Por ello se desactiva temporalmente:

```
contentSecurityPolicy: false
```

En producción se recomienda configurar CSP de acuerdo con las necesidades del sitio y no desactivarla completamente, por ahora para fines académicos simplificamos la configuración. Más adelante modificaremos esta cabecera.

#### Paso 4 – Reiniciar aplicación

```
pm2 restart dsw-a13-app
```



Verificar nuevamente la aplicación en el navegador web.

| Id | Nombre                      | Materia                    | Promedio |
|----|-----------------------------|----------------------------|----------|
| 1  | Sofía Martínez López        | Desarrollo de Sistemas Web | 9.2      |
| 2  | Miguel Ángel Torres Ruiz    | Enrutamiento Avanzado      | 8.5      |
| 3  | Carolina Herrera Gutiérrez  | Bases de Datos             | 9.8      |
| 4  | Laura González Sánchez      | Cómputo en la Nube         | 8        |
| 5  | Ricardo Fernández Moreno    | Enrutamiento Avanzado      | 7.8      |
| 6  | Ana María Sánchez Martín    | Arquitecturas de Red       | 9.5      |
| 7  | Juan Carlos Navarro Jiménez | Arquitecturas de Red       | 6.9      |
| 8  | Beatriz Molina Castro       | Sistemas Operativos        | 8.3      |
| 9  |                             |                            |          |
| 10 |                             |                            |          |
| 11 |                             |                            |          |

## Parte G. Agregar cabeceras de seguridad en Apache

### Paso 1 – Editar configuración Apache HTTPS

Abre el archivo de configuración de Apache creado de forma automática por Certbot durante la aplicación del certificado cuando ejecutaste `sudo certbot --apache`:

```
sudo nano /etc/apache2/sites-available/dsw-a14-le-ssl.conf
```

Agregar dentro de:

```
<VirtualHost *:443>
```

las siguientes cabeceras:

```
Header always set X-Content-Type-Options "nosniff"
Header always set X-Frame-Options "SAMEORIGIN"
Header always set Referrer-Policy "strict-origin-when-cross-origin"
Header always set Permissions-Policy "geolocation=(), microphone=(), camera=()"
Header always set Strict-Transport-Security "max-age=31536000; includeSubDomains"
```

Tu archivo debería de quedar similar al siguiente:

```
<IfModule mod_ssl.c>
<VirtualHost *:443>

    ServerName dsw-erika.eastus.cloudapp.azure.com

    ProxyPreserveHost On
```



```
ProxyPass / http://127.0.0.1:3000/  
ProxyPassReverse / http://127.0.0.1:3000/  
  
# Cabeceras de seguridad  
Header always set X-Content-Type-Options "nosniff"  
Header always set X-Frame-Options "SAMEORIGIN"  
Header always set Referrer-Policy "strict-origin-when-cross-origin"  
Header always set Permissions-Policy "geolocation=(), microphone=(), camera=()"  
Header always set Strict-Transport-Security "max-age=31536000; includeSubDomains"  
  
ErrorLog ${APACHE_LOG_DIR}/dsw-a14-error.log  
CustomLog ${APACHE_LOG_DIR}/dsw-a14-access.log combined  
  
SSLCertificateFile /etc/letsencrypt/live/dsw-erika.eastus.cloudapp.azure.com/fullchain.pem  
SSLCertificateKeyFile /etc/letsencrypt/live/dsw-erika.eastus.cloudapp.azure.com/privkey.pem  
Include /etc/letsencrypt/options-ssl-apache.conf  
</VirtualHost>  
</IfModule>
```

A continuación se muestra una explicación de estas importantes cabeceras:

X-Content-Type-Options: nosniff

- Evita que el navegador interprete incorrectamente archivos MIME.
- Mitiga ataques relacionados con contenido malicioso.

X-Frame-Options: SAMEORIGIN

- Impide que el sitio sea cargado en iframes externos.
- Ayuda contra clickjacking.

Referrer-Policy

- Controla cuánta información de navegación comparte el navegador.
- Protege privacidad del usuario.

Permissions-Policy

- Restringe acceso a APIs del navegador: Cámara, micrófono, geolocalización.

Strict-Transport-Security (HSTS)

- Fuerza uso exclusivo de HTTPS.
- Previene downgrade a HTTP inseguro.



## Paso 2 – Verificar configuración Apache

```
sudo apachectl configtest
```

## Paso 3 – Recargar Apache

```
sudo systemctl reload apache2
```

Revisa que tu sitio sigue funcionando correctamente en el navegador

## Parte H. Cerrar exposición directa de Node.js

### Paso 1 – Cerrar puerto 3000 en UFW

Se cerrará el puerto 3000 ya que no es necesario pues se ha implementado Apache como Proxy inverso. Solo Apache puede comunicarse con Node.js internamente:

```
azureuser@vm-dsw-a13-2-ErikaMeneses:/var/www/dsw-a13$ sudo ufw status
Status: active

To Action From
--
OpenSSH ALLOW Anywhere
80/tcp ALLOW Anywhere
3000/tcp ALLOW Anywhere
Apache Full ALLOW Anywhere
OpenSSH (v6) ALLOW Anywhere (v6)
80/tcp (v6) ALLOW Anywhere (v6)
3000/tcp (v6) ALLOW Anywhere (v6)
Apache Full (v6) ALLOW Anywhere (v6)
```

```
sudo ufw delete allow 3000/tcp
```

Verifica:

```
sudo ufw status
```

El puerto 3000 ya no debe aparecer.



```
[azureuser@vm-dsw-a13-2-ErikaMeneses:/var/www/dsw-a13$ sudo ufw delete allow 3000/tcp
Rule deleted
Rule deleted (v6)
[azureuser@vm-dsw-a13-2-ErikaMeneses:/var/www/dsw-a13$ sudo ufw status
Status: active

To Action From
--
OpenSSH ALLOW Anywhere
80/tcp ALLOW Anywhere
Apache Full ALLOW Anywhere
OpenSSH (v6) ALLOW Anywhere (v6)
80/tcp (v6) ALLOW Anywhere (v6)
Apache Full (v6) ALLOW Anywhere (v6)
```

## Paso 2 – Eliminar regla 3000 en Azure

En Azure:

1. VM → Networking
2. Eliminar regla:

Allow-Node-3000

| Buscar reglas                   |                                  | Origen == todo |            | Destino == todo |                   | Protocolo == todo |  | Acción == todo |  | Puerto == todo |  |
|---------------------------------|----------------------------------|----------------|------------|-----------------|-------------------|-------------------|--|----------------|--|----------------|--|
| Prioridad ↑                     | Nombre                           |                | Puerto     | Protocolo       | Origen            | Destino           |  | Acción         |  |                |  |
| Reglas de puerto de entrada (7) |                                  |                |            |                 |                   |                   |  |                |  |                |  |
| 300                             | SSH                              |                | 22         | TCP             | Cualquiera        | Cualquiera        |  | Allow          |  |                |  |
| 310                             | Allow-Node-3000                  |                | 3000       | TCP             | Cualquiera        | Cualquiera        |  | Allow          |  |                |  |
| 320                             | HTTP                             |                | 80         | TCP             | Cualquiera        | Cualquiera        |  | Allow          |  |                |  |
| 330                             | HTTPS                            |                | 443        | TCP             | Cualquiera        | Cualquiera        |  | Allow          |  |                |  |
| 65000                           | AllowVnetInBound ⓘ               |                | Cualquiera | Cualquiera      | VirtualNetwork    | VirtualNetwork    |  | Allow          |  |                |  |
| 65001                           | AllowAzureLoadBalancerInBou... ⓘ |                | Cualquiera | Cualquiera      | AzureLoadBalancer | Cualquiera        |  | Allow          |  |                |  |
| 65500                           | DenyAllInBound ⓘ                 |                | Cualquiera | Cualquiera      | Cualquiera        | Cualquiera        |  | Deny           |  |                |  |

Ahora Node.js debe seguir funcionando, escucha localmente y ya no es accesible desde Internet.

## Parte I. Personalizar cabecera `contentSecurityPolicy`

### Paso 1. Modificar en `index.js` la configuración de la cabecera Content Security Policy

En la Parte F. Implementar Helmet en Express, se configuró la cabecera de seguridad Content Security Policy (CSP) de manera laxa, a continuación se endurecerá esta configuración:

En `index.js` sustituye:



```
app.use(  
  helmet({  
    // Se desactiva CSP temporalmente  
    // porque Bootstrap y algunos recursos externos  
    // pueden ser bloqueados durante la práctica  
    contentSecurityPolicy: false  
  })  
);
```

Por:

```
app.use(  
  helmet({  
    contentSecurityPolicy: {  
      useDefaults: true,  
      directives: {  
        "default-src": ["'self'"],  
  
        "script-src": [  
          "'self'",  
          "https://cdn.jsdelivr.net",  
          "https://cdnjs.cloudflare.com"  
        ],  
  
        "style-src": [  
          "'self'",  
          "'unsafe-inline'",  
          "https://cdn.jsdelivr.net",  
          "https://cdnjs.cloudflare.com",  
          "https://fonts.googleapis.com"  
        ],  
  
        "font-src": [  
          "'self'",  
          "https://fonts.gstatic.com",  
          "https://cdn.jsdelivr.net",  
          "https://cdnjs.cloudflare.com",  
          "data:"  
        ],  
  
        "img-src": [  
          "'self'",  
          "data:",  
          "https:"  
        ],  
  
        "connect-src": [  
          "'self'",  
          "https://cdn.jsdelivr.net",  
          "https://cdnjs.cloudflare.com"  
        ],  
  
        "object-src": ["'none'"],  
  
        "frame-ancestors": ["'self'"],  
        "base-uri": ["'self'"],  
      }  
    }  
  })  
);
```





```
    "form-action": ["'self'"],  
    "upgrade-insecure-requests": []  
  }  
}  
})  
);
```

Todo tu index.js debe quedar de la siguiente manera:

```
const express = require('express');  
const helmet = require('helmet');  
const dotenv = require('dotenv');  
  
const app = express();  
  
// Carga la configuración del archivo .env  
dotenv.config();  
  
// Obtiene el puerto desde .env  
const PORT = process.env.SERVER_PORT || 3000;  
  
// Configura EJS como motor de vistas  
app.set('view engine', 'ejs');  
  
// Middleware Helmet  
// Agrega cabeceras HTTP de seguridad automáticamente  
app.use(  
  helmet({  
    contentSecurityPolicy: {  
      useDefaults: true,  
      directives: {  
        "default-src": ["'self'"],  
  
        "script-src": [  
          "'self'",  
          "https://cdn.jsdelivr.net",  
          "https://cdnjs.cloudflare.com"  
        ],  
  
        "style-src": [  
          "'self'",  
          "'unsafe-inline'",  
          "https://cdn.jsdelivr.net",  
          "https://cdnjs.cloudflare.com",
```



```
        "https://fonts.googleapis.com"
    ],

    "font-src": [
        "'self'",
        "https://fonts.gstatic.com",
        "https://cdn.jsdelivr.net",
        "https://cdnjs.cloudflare.com",
        "data:"
    ],

    "img-src": [
        "'self'",
        "data:",
        "https:"
    ],

    "connect-src": [
        "'self'",
        "https://cdn.jsdelivr.net",
        "https://cdnjs.cloudflare.com"
    ],

    "object-src": ["'none'"],

    "frame-ancestors": ["'self'"],

    "base-uri": ["'self'"],

    "form-action": ["'self'"],

    "upgrade-insecure-requests": []
    }
    })
);

// Middleware para procesar formularios
app.use(express.urlencoded({ extended: true }));

// Middleware para archivos JSON
app.use(express.json());

// Middleware para archivos estáticos
app.use(express.static('public'));
```



```
// Rutas
const homeRouter = require('./routes/home');

app.use('/', homeRouter);
app.use('/home', homeRouter);

// Middleware para manejo de errores
// Debe colocarse al final
const errorHandler = require('./middlewares/errorhandler');

app.use(errorHandler);

// Inicia el servidor web
app.listen(PORT, "0.0.0.0", () => {
  console.log(`Aplicación ejecutándose en el puerto ${PORT}`);
});
```

La cabecera Content-Security-Policy (CSP) permite controlar desde qué orígenes el navegador puede cargar recursos como scripts, hojas de estilo, fuentes, imágenes o conexiones externas. Su propósito principal es reducir el impacto de ataques como Cross-Site Scripting (XSS), evitando que el navegador ejecute código no autorizado.

En esta configuración se permite cargar recursos propios del sitio mediante `'self'` y se agregan dominios externos comunes utilizados en aplicaciones web reales, como:

<https://cdn.jsdelivr.net>  
<https://cdnjs.cloudflare.com>  
<https://fonts.googleapis.com>  
<https://fonts.gstatic.com>

A continuación una breve explicación de las principales directivas:

| Directiva                 | Función                                                                                                                                   |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| default-src<br>'self'     | Política base: solo recursos del mismo sitio                                                                                              |
| script-src                | Permite JavaScript local y CDN autorizados                                                                                                |
| style-src                 | Permite CSS local, Bootstrap CDN y Google Fonts                                                                                           |
| font-src                  | Permite fuentes externas autorizadas                                                                                                      |
| img-src                   | Permite imágenes locales, HTTPS y data:                                                                                                   |
| connect-src               | Permite peticiones fetch() al mismo sitio y a desde cdn.jsdelivr.net para archivos source maps (.map) usados por DevTools para depuración |
| object-src 'none'         | Bloquea objetos embebidos inseguros                                                                                                       |
| frame-ancestors<br>'self' | Evita que sitios externos incrusten la página                                                                                             |



# UNIVERSIDAD VERACRUZANA. Licenciatura en Ingeniería de Ciberseguridad e Infraestructura de Cómputo

## EE. Desarrollo de Sistemas Web

| Directiva                 | Función                                       |
|---------------------------|-----------------------------------------------|
| form-action<br>'self'     | Los formularios solo se envían al mismo sitio |
| upgrade-insecure-requests | Intenta actualizar HTTP a HTTPS               |

Se mantiene 'unsafe-inline' en style-src porque Bootstrap y algunos estilos en plantillas EJS pueden requerir estilos inline. En un entorno de producción más estricto, lo ideal es eliminar estilos inline y usar solo archivos CSS externos controlados por la aplicación.

### Paso 2 – Reiniciar aplicación

```
pm2 restart dsw-a13-app
```

Verificar nuevamente la aplicación en el navegador web sin errores en la consola.

Alumnos con materias

Lista de alumnos

| Id | Nombre                      | Materia                    | Promedio |
|----|-----------------------------|----------------------------|----------|
| 1  | Sofía Martínez López        | Desarrollo de Sistemas Web | 9.2      |
| 2  | Miguel Ángel Torres Ruiz    | Enrutamiento Avanzado      | 8.5      |
| 3  | Carolina Herrera Gutiérrez  | Bases de Datos             | 9.8      |
| 4  | Laura González Sánchez      | Cómputo en la Nube         | 8        |
| 5  | Ricardo Fernández Moreno    | Enrutamiento Avanzado      | 7.8      |
| 6  | Ana María Sánchez Martín    | Arquitecturas de Red       | 9.5      |
| 7  | Juan Carlos Navarro Jiménez | Arquitecturas de Red       | 6.9      |
| 8  | Beatriz Molina Castro       | Sistemas Operativos        | 8.3      |



## Parte J. Validaciones finales

### Paso 1 – Verificar servicios

Apache:

```
sudo systemctl status apache2
```

PM2:

```
pm2 status
```

### Paso 2 – Verificar puertos activos

```
sudo ss -tulnp
```

Debe aparecer:

- 80
- 443
- 3000 localmente

```
azureuser@vm-dsw-a13-2-ErikaMeneses:/var/www/dsw-a13$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-05-25 15:45:41 UTC; 3h 22min ago
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 5982 ExecReload=/usr/sbin/apachectl graceful (code=exited, status=0/SUCCESS)
  Main PID: 981 (apache2)
    Tasks: 55 (limit: 2245)
   Memory: 16.5M (peak: 19.3M)
      CPU: 1.309s
   CGroup: /system.slice/apache2.service
           └─ 981 /usr/sbin/apache2 -k start
             └─ 5987 /usr/sbin/apache2 -k start
               └─ 5988 /usr/sbin/apache2 -k start

May 25 15:45:39 vm-dsw-a13-2-ErikaMeneses systemd[1]: Starting apache2.service - The Apache HTTP Server...
May 25 15:45:41 vm-dsw-a13-2-ErikaMeneses systemd[1]: Started apache2.service - The Apache HTTP Server.
May 25 16:08:30 vm-dsw-a13-2-ErikaMeneses systemd[1]: Reloading apache2.service - The Apache HTTP Server...
May 25 16:08:30 vm-dsw-a13-2-ErikaMeneses systemd[1]: Reloaded apache2.service - The Apache HTTP Server.
May 25 17:41:46 vm-dsw-a13-2-ErikaMeneses systemd[1]: Reloading apache2.service - The Apache HTTP Server...
May 25 17:41:46 vm-dsw-a13-2-ErikaMeneses systemd[1]: Reloaded apache2.service - The Apache HTTP Server.
azureuser@vm-dsw-a13-2-ErikaMeneses:/var/www/dsw-a13$ pm2 status
```

| id | name        | namespace | version | mode | pid  | uptime | ♾ | status | cpu | mem    | user   | watching |
|----|-------------|-----------|---------|------|------|--------|---|--------|-----|--------|--------|----------|
| 0  | dsw-a13-app | default   | 1.0.0   | fork | 7003 | 3m     | 4 | online | 0%  | 68.7mb | azu... | disabled |

```
azureuser@vm-dsw-a13-2-ErikaMeneses:/var/www/dsw-a13$ sudo ss -tulnp
Netid    State      Recv-Q     Send-Q      Local Address:Port      Peer Address:Port      Process
udp      UNCONN    0           0           127.0.0.54:53            0.0.0.0:*               users:((("systemd-resolve",pid=543,fd=16))
udp      UNCONN    0           0           127.0.0.53:53            0.0.0.0:*               users:((("systemd-resolve",pid=543,fd=14))
udp      UNCONN    0           0           10.0.0.46:eth0:68        0.0.0.0:*               users:((("systemd-network",pid=734,fd=21))
udp      UNCONN    0           0           127.0.0.1:323            0.0.0.0:*               users:((("chronyd",pid=869,fd=6))
udp      UNCONN    0           0           [::]:323                 [::]:*                  users:((("chronyd",pid=869,fd=7))
tcp      LISTEN    0           511          0.0.0.0:3000             0.0.0.0:*               users:((("node /var/www/d",pid=7003,fd=26))
tcp      LISTEN    0           4096         127.0.0.54:53            0.0.0.0:*               users:((("systemd-resolve",pid=543,fd=17))
tcp      LISTEN    0           4096         127.0.0.53:53            0.0.0.0:*               users:((("systemd-resolve",pid=543,fd=15))
tcp      LISTEN    0           151         127.0.0.1:3006            0.0.0.0:*               users:((("mysqld",pid=976,fd=25))
tcp      LISTEN    0           4096         0.0.0.0:22                0.0.0.0:*               users:((("sshd",pid=1293,fd=3),("systemd",pid=1,fd=230))
tcp      LISTEN    0           70          127.0.0.1:3006           0.0.0.0:*               users:((("mysqld",pid=976,fd=21))
tcp      LISTEN    0           511          *:443                     *:.*                     users:((("apache2",pid=5988,fd=6),("apache2",pid=5987,fd=6),("apache2",pid=981,fd=6))
tcp      LISTEN    0           4096         [::]:22                   [::]:.*                  users:((("sshd",pid=1293,fd=4),("systemd",pid=1,fd=231))
tcp      LISTEN    0           511          *:80                      *:.*                     users:((("apache2",pid=5988,fd=4),("apache2",pid=5987,fd=4),("apache2",pid=981,fd=4))
```



### Paso 3 – Verificar cabeceras HTTPS

```
curl -I https://dsw-erika.eastus.cloudapp.azure.com/
```

Debes observar cabeceras como:

Strict-Transport-Security  
X-Frame-Options  
X-Content-Type-Options

```
azureserver@dsw-a13-2-ErikaMeneses:/var/www/dsw-a13$ curl -I https://dsw-erika.eastus.cloudapp.azure.com/
HTTP/1.1 200 OK
Date: Mon, 25 May 2026 19:11:31 GMT
Server: Apache/2.4.58 (Ubuntu)
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: geolocation=(), microphone=(), camera=()
Strict-Transport-Security: max-age=31536000; includeSubDomains
Content-Security-Policy: default-src 'self';script-src 'self' https://cdn.jsdelivr.net https://cdnjs.cloudflare.com;style-src 'self' 'unsafe-inline' https://cdn.jsdelivr.net https://cdnjs.cloudflare.com https://fonts.googleapis.com;font-src 'self' https://fonts.gstatic.com https://cdn.jsdelivr.net https://cdnjs.cloudflare.com data:;img-src 'self' data: https://connect-src 'self' https://cdn.jsdelivr.net https://cdnjs.cloudflare.com;object-src 'none';frame-ancestors 'self';base-uri 'self';form-action 'self';upgrade-insecure-requests;script-src-attr 'none'
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Origin-Agent-Cluster: ?1
Referrer-Policy: no-referrer
Strict-Transport-Security: max-age=31536000; includeSubDomains
X-Content-Type-Options: nosniff
X-DNS-Prefetch-Control: off
X-Download-Options: noopen
X-Frame-Options: SAMEORIGIN
X-Permitted-Cross-Domain-Policies: none
X-XSS-Protection: 0
Content-Type: text/html; charset=utf-8
Content-Length: 9692
ETag: W/"25dc-xXMONNqgB47f3u+BfKY0ocbM6gs"
```

### Paso 4 – Verificar acceso HTTPS

Abrir:

<https://dsw-erika.eastus.cloudapp.azure.com/>

La aplicación debe:

- Funcionar
- Usar HTTPS
- Mostrar candado
- No usar puerto 3000 externamente